INTERNSHIP PROJECT REPORT

AI RESPONSE VALIDATION SYSTEM WITH HALLUCINATION DETECTION ASSISTANCE

Submitted as part of the internship project

Abstract

The AI Response Validation System with Hallucination Detection Assistance is designed to help assess AI-generated responses in a structured way. The application accepts a question and an AI response, with optional reference answers or source documents. It evaluates responses across relevance, accuracy, hallucination risk, and completeness, then combines the available evaluation outputs into an overall verdict. The application supports single-response evaluation and CSV-based batch evaluation, and provides dashboard and history views for reviewing results. The implementation uses Python and Streamlit, with a modular agent structure, a knowledge-retrieval component, and SQLite storage. The system is intended to assist review rather than guarantee correctness; evaluation results should be checked against source evidence and test outcomes.

1. Introduction

Large language models can generate useful answers, but their outputs may be irrelevant, incomplete, inaccurate, or contain unsupported claims. Reviewing responses one by one can be difficult when the number of test cases grows. This project provides an application for evaluating AI responses using a repeatable multi-dimensional workflow.

1.1 Background

Response evaluation can consider whether an answer addresses the question, whether its claims are supported, whether it contains potentially fabricated information, and whether it covers the important parts of the request. The project organizes these checks into separate evaluation agents and combines their outputs for easier review.

1.2 Problem Statement

There is a need for a structured way to review AI-generated responses for relevance, accuracy, completeness, and potential hallucinations, including when multiple responses must be processed. Manual checking alone can be slow and inconsistent. The project aims to support this process through a modular application and batch workflow.

1.3 Objectives

- Provide a user interface for entering a question and AI-generated response.
- Allow optional reference answers or source documents to support evaluation.
- Assess relevance, accuracy, hallucination risk, and completeness using separate agents.
- Combine evaluation outputs into an overall verdict.
- Support single and CSV batch evaluation.
- Store and display evaluation history and dashboard summaries.

- Support evaluation through configured model-provider options.

2. Scope of the Project

The project focuses on model-assisted evaluation of text responses. It includes a Streamlit interface, Python orchestration, separate evaluation-agent modules, knowledge retrieval where configured, local SQLite persistence, and batch processing. It does not independently prove the truth of every claim. The reliability of the output depends on the model provider, prompts, input quality, reference material, and evaluation logic.

3. System Requirements

3.1 Functional Requirements

- Accept a question and AI response as input.
- Accept optional reference answer and/or reference document.
- Run the configured evaluation agents.
- Generate an overall verdict from evaluation outputs.
- Support multiple test cases through a CSV batch input.
- Display results in the interface and provide access to saved history/dashboard views.
- Handle invalid inputs and provider failures as supported by the implementation.

3.2 Non-Functional Requirements

- Maintain a modular code structure so evaluation components can be updated independently.
- Provide a clear interface for submitting inputs and reviewing results.
- Keep API credentials out of source control.
- Make evaluation behavior testable with repeatable input cases.
- Provide clear errors when required input or provider configuration is unavailable.

4. Technologies Used

Technology	Role in the project
Python	Application logic and evaluation orchestration
Streamlit	Interactive interface, dashboard, history, and input screens
SQLite	Local storage for evaluation records and batch-related data
FAISS	Vector similarity search in the knowledge-retrieval component
Sentence Transformers	Creating embeddings for retrieval
Groq API / Gemini API / Ollama	Model-provider options available in the project configuration
Pandas and CSV	Handling batch evaluation input and tabular data
PDF generation package	Report export, if enabled in the final application

The final package versions should be documented from the tested requirements.txt. Exact availability of each provider and export feature depends on the current code and deployment configuration.

5. System Design and Architecture

The application follows a modular design. The user interface collects inputs and displays outputs. The evaluation orchestrator coordinates the process and calls individual agents. A retrieval component can provide relevant indexed reference material. A verdict component combines the evaluation outputs, while SQLite stores records for later review.

1. Input: question, AI response, and optional reference answer/document.
2. Interface: Streamlit screens for single evaluation, batch evaluation, history, and dashboard.
3. Orchestrator: controls the evaluation sequence and coordinates agent execution.
4. Evaluation agents: assess relevance, accuracy, hallucination risk, and completeness.
5. Knowledge retrieval: retrieves relevant indexed chunks when configured and available.
6. Verdict generation: combines agent outputs according to the configured project logic.
7. Storage: SQLite records evaluation data and batch-related information.
8. Results: the interface presents scores, verdicts, history, dashboard summaries, and available exports.

6. Module Description

- **app.py:** Main Streamlit application and user-facing screens.
- **orchestrator.py:** Coordinates the evaluation process and agent calls.
- **database.py:** Initializes and interacts with SQLite storage.
- **agents/relevance_agent.py:** Evaluates whether the response addresses the question.
- **agents/accuracy_agent.py:** Assesses accuracy using the available input and evidence.
- **agents/hallucination_agent.py:** Assesses potentially unsupported or fabricated claims.
- **agents/completeness_agent.py:** Assesses coverage of key aspects of the question.
- **agents/verdict_agent.py:** Combines available evaluation outputs into an overall verdict.
- **kb/:** Contains knowledge-base loading, preprocessing, chunking, embeddings, indexing, and retrieval modules.
- **batch_evaluator.py:** Supports processing multiple evaluation cases.

7. Methodology

The system follows a sequence of input validation, optional evidence retrieval, multi-agent assessment, verdict generation, and result presentation. Each agent focuses on one evaluation dimension. The orchestrator gathers the agent outputs, and the verdict component applies the configured combination rules. Because the exact scoring weights and thresholds are implementation-specific, they should be copied into the final report only after checking the latest code.

1. Collect the evaluation inputs.
2. Check that required fields are present.
3. Retrieve reference chunks when the retrieval path is enabled.
4. Run the configured evaluation agents.
5. Collect and validate agent outputs.
6. Calculate the overall assessment and verdict using the implementation's rules.
7. Save results and update the user interface where supported.

8. Single and Batch Evaluation

8.1 Single Evaluation

Single evaluation allows the user to submit one question and response, optionally with reference material. The system runs the evaluation workflow and presents the resulting dimension scores and overall verdict.

8.2 Batch Evaluation

Batch evaluation processes multiple test cases from a CSV file using the column names expected by the application. This makes it possible to evaluate a set of responses in one workflow. Invalid rows and provider errors should be reviewed in the batch output; exact behavior should be confirmed against the latest version.

9. Data Storage and Results

SQLite is used for local persistence of evaluation information. The dashboard and history views help review prior evaluations and aggregate results. When running on a hosted platform, the persistence behavior of a local SQLite file must be checked because temporary application storage may not survive restarts or redeployments.

10. Testing Strategy

Testing should cover valid inputs, missing required fields, reference material, batch processing, agent output handling, database updates, provider configuration, and export functions. The repository contains selected test modules, but the report should include only test results that were actually executed and observed.

Test area	Expected behavior	Actual result
Valid single evaluation	Returns evaluation results	Evaluation completed successfully, and scores with an overall verdict were generated for valid inputs.
Missing AI response	Shows validation error or handles invalid input	The missing response was identified as an invalid input during batch testing; the affected row was not evaluated successfully.
CSV with valid rows	Processes cases and presents results	Batch evaluation processed valid rows and generated evaluation results.
CSV with invalid row	Reports or skips invalid input according to implementation	Invalid rows were handled as errors, while valid rows in the batch were processed successfully.
Provider unavailable	Shows a handled error or fallback if implemented	The application handled the unavailable provider and displayed an error message.
History/dashboard	Displays saved or aggregated results	Evaluation history and dashboard displayed stored evaluation records, summary metrics, and evaluation scores.

11. Security and Deployment

- Keep .env, API keys, and secrets out of GitHub.
- Configure credentials through the hosting platform's secret-management feature.
- A hosted application cannot directly use an Ollama service running only on the developer's computer.
- Ensure the deployed environment installs the dependencies listed in requirements.txt.
- Include required source and knowledge-base files, and verify their size and availability.
- Confirm database persistence requirements before using the application for ongoing shared evaluations.

12. Limitations

- Model-based evaluations can be inconsistent or incorrect.
- An overall score is not a substitute for human review of important claims.
- Results depend on the quality and relevance of supplied references and retrieved material.
- Provider rate limits, network issues, or invalid credentials can interrupt evaluation.
- Hosted storage and local model access require separate deployment consideration.

13. Future Scope

- Validate scores against a human-reviewed benchmark set.
- Provide claim-level evidence and clearer explanations for flagged content.
- Expand tests for edge cases and provider failures.
- Add durable database storage for deployed use.
- Improve batch progress reporting, retry behavior, and error summaries.
- Document repeatable experiments across model providers.

14. Conclusion

The AI Response Validation System with Hallucination Detection Assistance organizes AI-response assessment into a modular workflow with relevance, accuracy, hallucination-risk, and completeness checks. Its single and batch interfaces, retrieval component, database, and dashboard/history views are intended to make response review more systematic. The project should be evaluated using documented test cases, and the final report should reflect only behavior verified in the submitted source code and deployed application.

References

- Project source code and README in the project's GitHub repository.
- Streamlit documentation: https://docs.streamlit.io/
- FAISS documentation: https://github.com/facebookresearch/faiss
- Sentence Transformers documentation: https://www.sbert.net/
- SQLite documentation: https://www.sqlite.org/docs.html